

Retrospective Sprint 3 of Group UG13

Block Model Compression Algorithm

Team Members:

1. a1837374 Mengbo Ren
2. a1837306 Suchang Liu
3. a1824243 Yuyao Chen
4. a1837254 Yang Shi
5. a1832623 Hin Kai Chan
6. a1860970 Mohsan Salehi-Pugsley
7. a1818124 Chih-Hao Chang

Snapshots

I attended the sprint review/planning meeting on the 2/Oct with the tutor.

Snapshots are provided in the appendix at the end of the retrospective.

What went well in the sprint?

Based on the latest client's feedback, we refined the task board to make it easier for the client to track our progress. Each issue now includes more detailed information, progress updates through comments, and links to relevant website for quick reference. These changes can promote more valuable feedback and client involvement. For example, after reviewing our product backlog, the client can quickly understand our plan and suggested the next focus for our team.

Detailed progress updates also improved team communication and made it more transparent. If a team member encounters a task dependency issue, they can quickly access relevant task information without consulting others directly. Overall, these enhancements promoted better collaboration and led to a higher-quality final product by integrating teamwork and user feedback.

Additionally, in the third sprint, we streamlined our workflow using an approach of research, implement and test. Tasks now have clearer boundaries and fewer dependencies, increasing efficiency and quality of work. For instance, when implementing multi-threading in Python to increase the execution speed, I was stuck due to the limitations of language's high-level nature. However, I was able to re-implement the solution in C++ using this approach, and I boosted both the compression and speed on the Titan testing machine.

Moreover, project management improvements have led to more productive team meetings. We've shifted from discussing individual solutions to focusing on the project's broader scope. For example, we now prioritize discussions on how components like run-length encoding and Octree structures interact and leverage threading. This has helped us quickly identify system bottlenecks and manage time more effectively, keeping the project on track.

What could be improved?

Firstly, I believe the task analysis process needs to be improved to allocate resources more effectively. Assigning team members based solely on task difficulty is not enough. The client was confused as to why multiple people were assigned to a single task. While task difficulty is an important factor, we also need to demonstrate that the complex task can be worked on in parallel, allowing multiple members to contribute without downtime or conflict. This approach will lead to more efficient resource management and a higher-quality outcome. If a task cannot be proven parallelizable, it's likely for the points to congest that needs to be resolved, allowing

the team to address issues quickly. This refinement will also improve client satisfaction, as it shows that their investment is being used efficiently without unnecessary overheads like idle time or team friction.

Secondly, a key area for improvement in the next sprint is staying focused on the main goals for better time management and solution compatibility. The client noticed that the product backlog lacked ideas and updates for the next phase (such as optimization). Since each sprint builds on the previous work, neglecting the planning of next phase could result in solutions that don't align well with future needs. By planning with the next phase in mind, the team will stay on track, ensuring smoother transitions and better time control to result in more compatible solutions between phases, making the software more stable and reliable.

Lastly, as the project nears completion, the team should dedicate more time to improve the software's maintainability and scalability after finalizing the implementation and testing of working code. This includes organizing the code modularly, refactoring, enhancing readability, and providing thorough documentation throughout development. Doing so will not only boost client satisfaction but also make it easier for them to add future features and maintain transparency within the software.

What will the group commit to improve in the next sprint?

To improve efficiency in our sprint planning, we've decided to incorporate task parallelization as part of task analysis. The scrum master will assess whether complex tasks can be parallelized before assigning additional team members. A detailed discussion between the scrum master and the small team handling the task will help create a clear plan, assigning subtasks to ensure smooth collaboration and minimize idle time. The scrum master will then document the approach for client review. If a task cannot be effectively parallelized, we will reanalyze it, breaking it down into smaller, parallelizable components.

Additionally, we are committed to outlining key considerations for future sprints during current sprint planning. These will be discussed by the team and added to the product backlog, guiding the solution design to be both complete and aligned with long-term goals, ensuring stable and consistent deliverables.

Starting next sprint, one team member has been assigned to review the existing codebase for adherence to coding standards and readability, while another team member will handle software usage, consulting both the development and testing sub-teams. This approach will not only prepare us for the optimization phase but will also ensure the software remains maintainable, scalable, and future-proof, while improving user experience.

My progress this sprint

Implement simple compression algorithm #26

a1818124 opened last month

a1818124 last month

Edit

Assignees a1818124

Labels Add labels...

Milestone Add milestone...

Status **Sprint 3 Done**

Linked pull requests No linked pull requests

Repository BLOCK-UG13-SEP-2024

Priority **P1**

Size **L**

Estimate 10

Iteration sprint 3

Write

Preview

H B I

Leave a comment

Description: Implement a simple compression algorithm after establishing the algorithm used.

Definition of Done: Algorithm produces expected output.

Integrate octree data structure #42

a1818124 opened 13 minutes ago

a1818124 now (edited)

Edit

Assignees a1837306, a1837374, and a18...

Labels Add labels...

Milestone Add milestone...

Status **Sprint 3 Done**

Linked pull requests No linked pull requests

Repository BLOCK-UG13-SEP-2024

Priority **P0**

Size **M**

Estimate 4

Iteration sprint 3

Write

Preview

H B I

Leave a comment

Description: integrate octree data structure to existing compression algorithm

DoD: working compression code using octree algorithm

Purpose: improved speed of lookups and compression

Obstacles: merging components near each other (maintain a suitable data structure to track the details of each rectangle as it is processed).

Implement alternative greedy algorithm #41

a1818124 opened 49 minutes ago

a1818124 15 minutes ago (edited)

Edit

Assignees a1860970 and a1818124

Labels Add labels...

Milestone Add milestone...

Status **Sprint 3 Done**

Linked pull requests No linked pull requests

Repository BLOCK-UG13-SEP-2024

Priority **P2**

Size **S**

Estimate 2

Iteration sprint 3

Write

Preview

H B I

Leave a comment

Def of Done: Store coordinates of connected pairs within the rolling window using a set.

Outcome: enhanced compression performance as the consumption of extra memory is reduced by keeping the pairs of different kinds in different buffers.

Research multi-threading implementation #43

a1818124 opened now

a1818124 now

Edit

Assignees a1818124

Labels Add labels...

Milestone Add milestone...

Status **Sprint 3 Done**

Linked pull requests No linked pull requests

Repository BLOCK-UG13-SEP-2024

Priority **P2**

Size **M**

Estimate 2

Iteration sprint 3

Write

Preview

H B I

Leave a comment

Description: research how to utilise parallel programming techniques (multi-threading specifically) to improve code execution speed.

Definition of Done: have a blueprint of how to intergrate multi-threading into the code.

Obstacles: GIT mutex blocks multi-threading. -> Current Solution: C++ is preferred as it provides more low level control over thread.

My main goal in this sprint is performance optimization, including the optimization of the compression ratio. I have integrated alternative algorithms and data structures for that purpose and have implemented multithreading to accelerate third-dimension compression.

First, I work on the implementation of an alternative greedy algorithm which could enhance the compression performance as the consumption of extra memory is reduced by keeping the pairs of different kinds in different buffers, thus allowing faster lookups and removals. Testing on memory usage and performance was performed with diverse inputs. The results showed less memory usage than Octree and better performance than simple run length encoding.

Next, we employed Octree data structure for efficient optimization of spatial indexing and querying while compressing in the third dimension. To do so, I first added input validation code to construct Octree. Then, I integrated the existing compression algorithm with additional code to perform operations like insertions, deletions, and lookups on the Octree structure. This has resulted in tremendous performance gains for our run-length encoding and block expanding algorithm.

Since I had some obstacles implementing multi-threading in Python as CPython interpreter only runs one thread at a time, I decided to implement multi-threading with C++ instead. At each step, thread scheduling and synchronization have been considered to improve the performance. Dynamic work distribution mechanism was used to avoid any thread's idle time (timer was utilized so that threads that finished their task could continue others task from the queue).

Appendix

Snapshot Week 7 of Group **Block-UG13**

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

a1837254 Yang SHI

a1832623 Hin Kai Chan

a1860970 Mohsan Salehi-Pugsley

a1818124 Chih-Hao Chang

a1881772 Nauman Yawar Butt

-

Product Backlog and Task Board:

The product backlog

Product backlog

6 / 3

Estimate: 14

This item hasn't been started

Draft

c++ related library

P2

S

3

sprint 2

Draft

Evaluation

P2

L

4

sprint 2

Draft

Methods & parameters

P2

M

3

sprint 2

Draft

Code examples

P2

M

4

sprint 3

BLOCK-UG13-SEP-2024 #25

Remove invisible characters from csv files

Draft

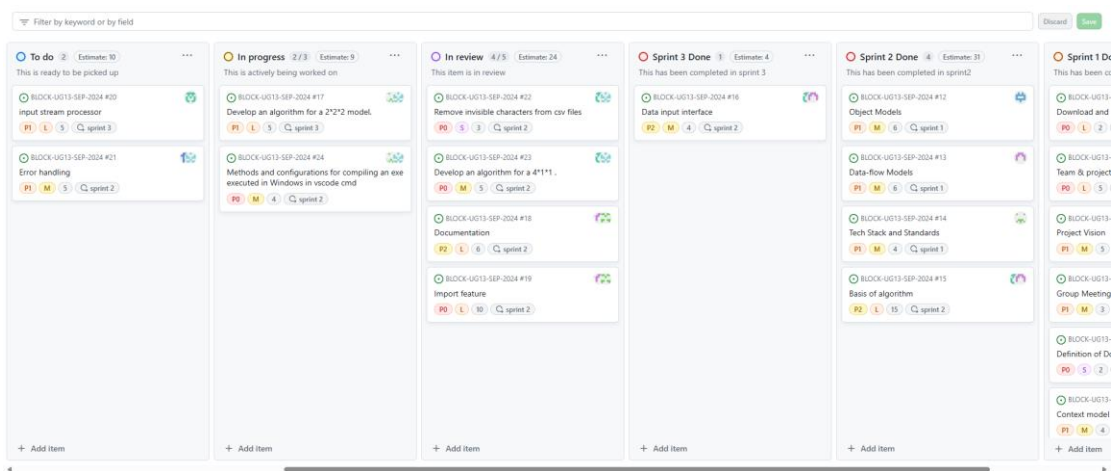
Develop an algorithm for a $n \times 1$

You can use

Control + Space

 to add an item

The task board



Sprint Backlog and User Stories:

Sprint backlog

 **Sprint Backlog**

3

Estimate: 5

...

 BLOCK-UG13-SEP-2024 #25

Remove invisible characters from csv files

 Draft

Develop an algorithm for a $n \times 1 \times 1$

 BLOCK-UG13-SEP-2024 #3



Optimize the input of data

P0

M

5

 sprint 3

 Add item

user stories

Sprint Overview

For the current sprint, we have selected the following user stories to focus on:

1. As a developer, looking up `line.empty()` during debugging results in an error.

Although the csv file looks like a blank line, it always doesn't work when determining conditions, resulting in errors.

2. As a developer, in order to familiarize yourself with the testing process, first implement a simple full version with compression capabilities in one direction.

3. Optimize data entry based on the results of previous sprints. A. File data.CVS input and stream; B. Check whether the printed data is consistent with the input data. C. By increasing the amount of data in the csv document, the test can run properly in stream mode.

These user stories and their related tasks will guide our development efforts for the current sprint, ensuring that we address the needs of both developers and clients in creating an effective block model compression program.

Definition of Done

1 Code complete:

Complete the code for the above three user use cases

2 Tests passed:

The code passed the local test, conforming to logic and the output met expectations.

Summary of Changes:

1. Modified the form of the runtime data file name as a parameter;
2. Checked and deleted the characters that could not be printed and displayed in the data file, and corrected the process error;
- 3, change the compilation environment cmd, and install MinGW compiler, so that exe can run normally under windows

Snapshot Week 8 of Group Block-UG13

Block Model Compression Algorithm

Team members:

a1837374 Mengbo Ren

a1837306 Suchang Liu

a1824243 Yuyao Chen

a1837254 Yang SHI

a1832623 Hin Kai Chan

a1860970 Mohsan Salehi-Pugsley

a1818124 Chih-Hao Chang

a1881772 Nauman Yawar Butt

Product Backlog and Task Board:

The product backlog

Product backlog 4 / 3



Estimate: 14

This item hasn't been started

 Draft ...



New features

P2

S

3

 sprint 3

 Draft



Enhancement

P2

L

4

 sprint 3

 Draft



Technical Debt

P2

M

3

 sprint 3

 Draft



Bug Fixes

P2

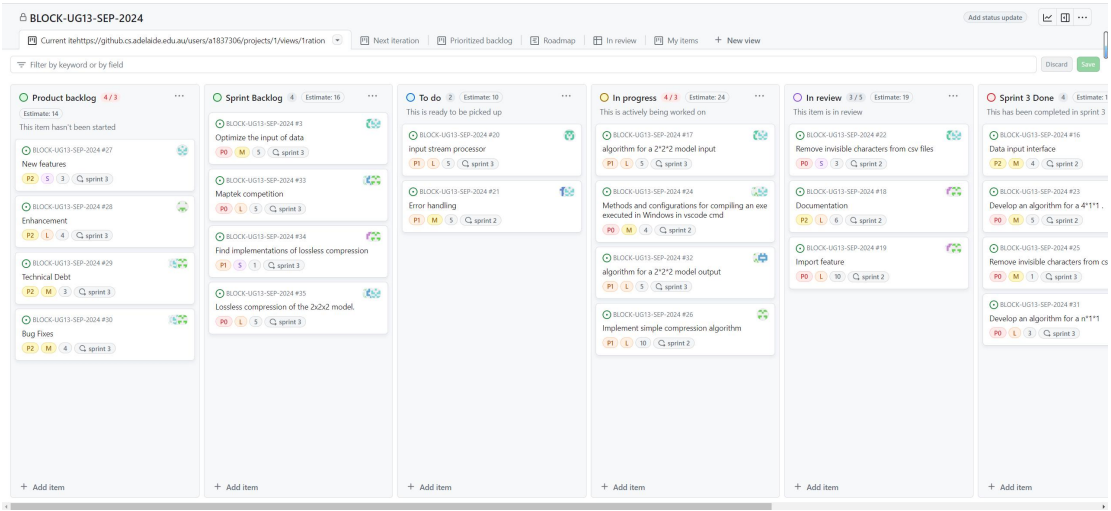
M

4

 sprint 3

+ Add item

The task board



Sprint Backlog and User Stories:

Sprint backlog

 Sprint Backlog 4 Estimate: 16 ...

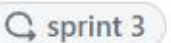
 BLOCK-UG13-SEP-2024 #3 

Optimize the input of data

  5 

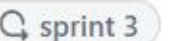
 BLOCK-UG13-SEP-2024 #33 

Maptek competition

  5 

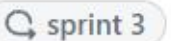
 BLOCK-UG13-SEP-2024 #34 

Find implementations of lossless compression

  1 

 BLOCK-UG13-SEP-2024 #35 ... 

Lossless compression of the 2x2x2 model.

  5 

+ Add item

User Stories

Sprint Overview

For the current sprint, we have selected the following user stories to focus on:

1. As a developer, looking up `line.empty()` during debugging results in an error. Although the csv file looks like a blank line, it always doesn't work when determining conditions, resulting in errors.
2. As a developer, in order to familiarize yourself with the testing process, first implement a simple full version with compression capabilities in one direction.
3. Optimize data entry based on the results of previous sprints. A. File data.CVS input and stream; B. Check whether the printed data is consistent with the input data. C. By increasing the amount of data in the csv document, the test can run properly in stream mode.

These user stories and their related tasks will guide our development efforts for the current sprint, ensuring that we address the needs of both developers and clients in creating an effective block model compression program.

Definition of Done

1 Code complete:

All planned functionality has been implemented, and the code conforms to the team's coding specifications.

2 Code Review passed:

The code has been peer reviewed at least once and all issues raised have been addressed.

3 Compile pass:

Compile through, there are no errors reported, and there are no warnings that affect the operation.

4 Tests passed:

All relevant automated tests (e.g., unit tests, integration tests) pass, and test coverage meets the standards agreed upon by the team.

All relevant manual tests (e.g., functional tests, acceptance tests) passed and no new defects were found.

5 Performance tests passed:

If applicable, all relevant performance tests have been passed and the performance metrics meet the standards agreed upon by the team.

6 Documentation updates:

All relevant technical documentation (such as design documentation, API documentation) has been updated and is consistent with the code.

7 Code merged into the main branch:

The code has been merged into the main branch and no new conflicts or problems have been introduced.

Summary of Changes:

1. Remove invisible characters from csv files
2. The RLE compression method was used for the 4*1*1 model, and the data was traversed. When encountering the same value, the count was recorded, and when the value changed, the current value and the count pair were stored in the result.
3. Try using the Quantization of Vertex Positions method to compress 3D models without loss of quality. This algorithm is still being refined.